05506026 Digital Image Processing (Java)

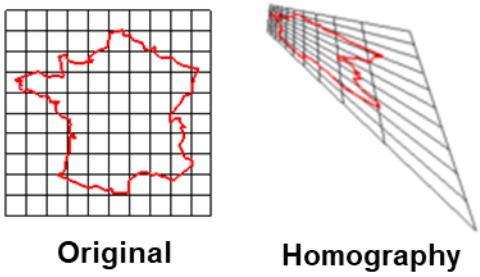
Lab 13

Objectives

1. Understand how to calculate homography transformation matrix.
2. Understand how to apply the homography transformation matrix to correct the perspective of images (image warping).

Homography transformation

Homography transformation is one of geometric transformation where the lines in an image are maintained. It is used regularly in the 2-dimentional image which captures the real 3-dimensional world that involves depth in the image.



The homography transformation, like the other 2-dimensional transformation, needs to use 3 x 3 transformation matrix.

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

Any points in one image will be mapped to the corresponding point in another image using this one matrix.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

From this matrix multiplication, we can conclude in algebra form that.

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}$$

Which can be reorganised as: -

$$x' = h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx'$$

$$y' = h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy'$$

Since there are 8 unknowns, we need 8 known x, y, x', and y' to solve this problem – which equals to 4 pairs of points.

In programming, we can also calculate this homography matrix. We start with using an image of QR code (qrcode.bmp). The objective is to remap the perspective of the code into perpendicular flat form.

The position of 4 pairs of corresponding corners are as follows: -

DIPLab.java

```
class DIPLab
{
  …
    public static void main (String[] args)

    {
        ...
        im.read("images/qrcode.bmp");
        double[][] srcPoints =
        {
            {256, 133}, // top-left
            {419, 146},  // top-right
            {403, 348}, // bottom-right
            {244, 320}  // bottom-left
        };

        double[][] dstPoints =
        {
            {0, 0},      // top-left
            {512, 0},    // top-right
            {512, 512},  // bottom-right
            {0, 512}     // bottom-left
        };
        ...
```

To solve the unknowns and form the homography transformation matrix, we can calculate in the form of $Ax = b$ which can be solved using any linear systems method. We will use the simplest method which is Gaussian elimination.

ImageManager.java

```
class ImageManager
{
  …
    public double[] calculateHomography(double[][] srcPoints, double[][] dstPoints)
    {
        double[][] A = new double[8][8];
        double[] b = new double[8];
```

```
        for (int i = 0; i < 4; i++) {
            double xSrc = srcPoints[i][0];
            double ySrc = srcPoints[i][1];
            double xDst = dstPoints[i][0];
            double yDst = dstPoints[i][1];

            A[2 * i][0] = xSrc;
            A[2 * i][1] = ySrc;
            A[2 * i][2] = 1;
            A[2 * i][3] = 0;
            A[2 * i][4] = 0;
            A[2 * i][5] = 0;
            A[2 * i][6] = -xSrc * xDst;
            A[2 * i][7] = -ySrc * xDst;

            A[2 * i + 1][0] = 0;
            A[2 * i + 1][1] = 0;
            A[2 * i + 1][2] = 0;
            A[2 * i + 1][3] = xSrc;
            A[2 * i + 1][4] = ySrc;
            A[2 * i + 1][5] = 1;
            A[2 * i + 1][6] = -xSrc * yDst;
            A[2 * i + 1][7] = -ySrc * yDst;

            b[2 * i] = xDst;
            b[2 * i + 1] = yDst;
        }
        // Solve using Gaussian elimination
        // This function will solve the system A * x = b
        // You can use Gaussian elimination, LU decomposition, or any other method
        return gaussianElimination(A, b);
    }

    public double[] gaussianElimination(double[][] A, double[] b)
    {
        int n = b.length;
        for (int i = 0; i < n; i++)
        {
            // Pivoting
            int max = i;
            for (int j = i + 1; j < n; j++)
            {
                if (Math.abs(A[j][i]) > Math.abs(A[max][i]))
                {
                    max = j;
```

```
                }
            }

            // Swap rows in A
            double[] temp = A[i];
            A[i] = A[max];
            A[max] = temp;

            // Swap entries in b
            double t = b[i];
            b[i] = b[max];
            b[max] = t;

            // Normalize the row
            for (int k = i + 1; k < n; k++)
            {
                double factor = A[k][i] / A[i][i];
                b[k] -= factor * b[i];
                for (int j = i; j < n; j++)
                {
                    A[k][j] -= factor * A[i][j];
                }
            }
        }

        // Back substitution
        double[] x = new double[n];
        for (int i = n - 1; i >= 0; i--)
        {
            double sum = 0.0;
            for (int j = i + 1; j < n; j++)
            {
                sum += A[i][j] * x[j];
            }
            x[i] = (b[i] - sum) / A[i][i];
        }

        // The last element of the homography matrix (h33) is 1
        double[] homography = new double[9];
        System.arraycopy(x, 0, homography, 0, 8);
        homography[8] = 1;

        return homography;
    }
```

The homography transformation matrix using 4 pairs of corners of the qrcode.bmp above should be solved as follows: -

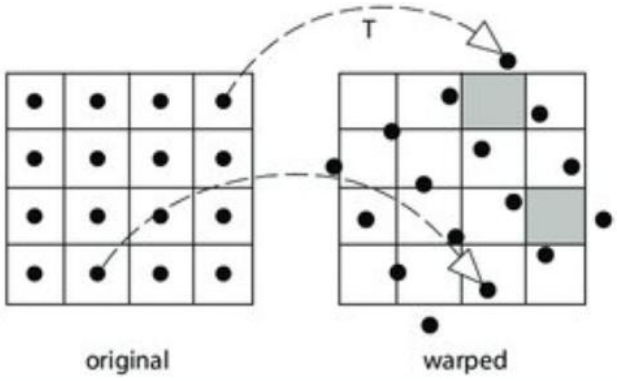
```
[3.862918370680192, 0.24788780988322132, -1021.8761816085976, -0.24370385208527345,
3.0556713761461203, -344.016106893604, 5.867969901251535E-4, -6.697009776615826E-5,
1.0]
```

After that, this matrix will be used to calculate the corresponding points from the source to destination.

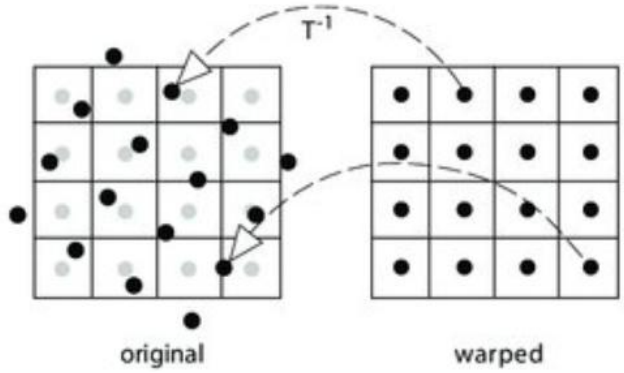
Image warpping

There are two types of perspective calculations – forward mapping and backward mapping.

Forward mapping is to map each pixel from the source image to its new location in the destination image. The disadvantage of this method is that we might end up with gaps or overlaps because the transformed points may not align perfectly on the grid of destination image pixels.



Backward mapping is to calculate from the destination image pixels. This allows us to iterate over every pixel in the destination image, determine where that pixel came from in the source image, and then assign the corresponding colour. This approach ensures that every pixel in the destination image is filled appropriately and avoids gaps. The only disadvantage of this approach is we need to use the inverse of the homography transformation matrix which cause additional calculations, and some matrix might not be invertible.



In this lab, we will use the backward mapping. We will start by creating a function to calculate the inverse of matrix.

ImageManager.java

```
class ImageManager
{
    ...
    public static double[] invertHomography(double[] H)
    {
        double[] invH = new double[9];

        double det = H[0] * (H[4] * H[8] - H[5] * H[7])
                   - H[1] * (H[3] * H[8] - H[5] * H[6])
                   + H[2] * (H[3] * H[7] - H[4] * H[6]);

        if (det == 0) throw new IllegalArgumentException("Matrix is not invertible");

        double invDet = 1.0 / det;
        invH[0] = invDet * (H[4] * H[8] - H[5] * H[7]);
        invH[1] = invDet * (H[2] * H[7] - H[1] * H[8]);
        invH[2] = invDet * (H[1] * H[5] - H[2] * H[4]);

        invH[3] = invDet * (H[5] * H[6] - H[3] * H[8]);
        invH[4] = invDet * (H[0] * H[8] - H[2] * H[6]);
        invH[5] = invDet * (H[2] * H[3] - H[0] * H[5]);

        invH[6] = invDet * (H[3] * H[7] - H[4] * H[6]);
        invH[7] = invDet * (H[1] * H[6] - H[0] * H[7]);
        invH[8] = invDet * (H[0] * H[4] - H[1] * H[3]);

        return invH;
    }
```

For calculating a corresponding point from another point using homography transformation matrix, we will implement as follows: -

ImageManager.java

```
class ImageManager
{
    ...
    public double[] applyHomographyToPoint(double[] H, double x, double y)
    {
        // Homogeneous coordinates calculation after transformation
        double xh = H[0] * x + H[1] * y + H[2];
        double yh = H[3] * x + H[4] * y + H[5];
        double w = H[6] * x + H[7] * y + H[8];

        // Normalize by w to get the Cartesian coordinates in the destination image
        double xPrime = xh / w;
        double yPrime = yh / w;

        return new double[]{xPrime, yPrime};
    }
```

This can be combined into another method where we iterate over every pixel of destination image and calculate back that which colour should be placed from the source image.

ImageManager.java

```
class ImageManager
{
    ...
    public void applyHomography(double[] H)
    {
        BufferedImage output = new BufferedImage(width, height, img.getType());

        double[] invH = invertHomography(H);

        // Iterate over every pixel in the destination image
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                 // Apply the inverse of the homography to find the corresponding source pixel
                 double[] sourcePoint = applyHomographyToPoint(invH, x, y);

                int srcX = (int) Math.round(sourcePoint[0]);
                int srcY = (int) Math.round(sourcePoint[1]);

                // Check if the calculated source coordinates are within the source image
bounds
                if (srcX >= 0 && srcX < width && srcY >= 0 && srcY < height)
                {
                    // Copy the pixel from the source image to the destination image
                    Color color = new Color(img.getRGB(srcX, srcY));
                    output.setRGB(x, y, color.getRGB());
                }
                else
                {
                    // If out of bounds, set the destination pixel to a default color
                    output.setRGB(x, y, Color.BLACK.getRGB());
                }
            }
        }
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                img.setRGB(x, y, output.getRGB(x, y));
            }
        }
    }
```

Quest 017

In the example of lab 13, the QR code is not in the correct orientation (3 big squares are at the top-left, top-right, and bottom-left corners of the QR code). Re-calculate the homography transformation matrix of the correct orientation.

The player who finishes this task will be awarded **15 EXP.**

Deadline: 7 September 2025

Quest 018

Use the homography transformation matrix in quest 017 to correct the orientation of the QR code.

The player who finishes this task will be awarded **25 EXP and 2 moves.**

Deadline: 7 September 2025